

第01章：LangChain使用概述

讲师：尚硅谷-宋红康

官网：[尚硅谷](#)



LangChain

release [langchain-core==0.3.72](#) CI [passing](#) license [MIT](#) downloads [59M/month](#) stars [112k](#) open issues [187](#)
Dev Containers [Open](#) Open in GitHub Codespaces Follow @LangChainAI [codspeed](#)

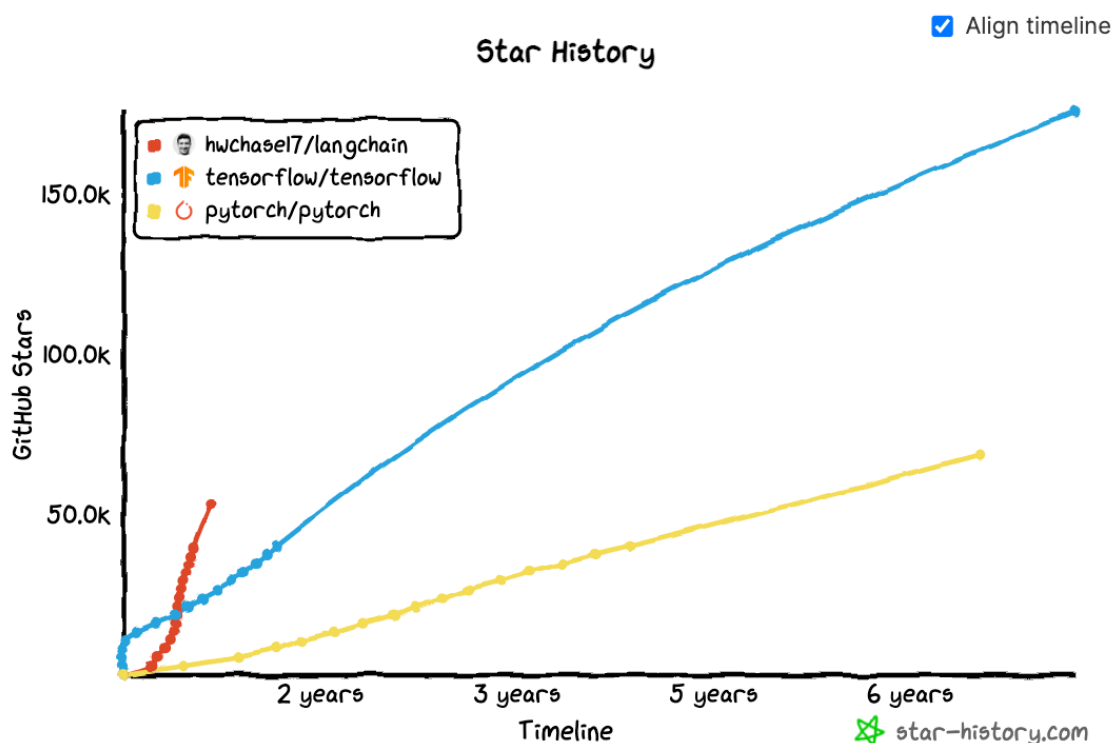
1、介绍LangChain

1.1 什么是LangChain

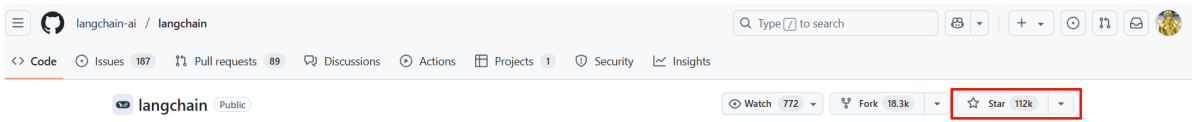
LangChain是 2022年10月，由哈佛大学的 **Harrison Chase**（哈里森·蔡斯）发起研发的一个开源框架，用于开发由大语言模型（LLMs）驱动的应用程序。

比如，搭建“智能体”（Agent）、问答系统（QA）、对话机器人、文档搜索系统、企业私有知识库等。

LangChain在Github上的热度变化



LangChain在Github上的star



<https://github.com/langchain-ai/langchain>

简单概括

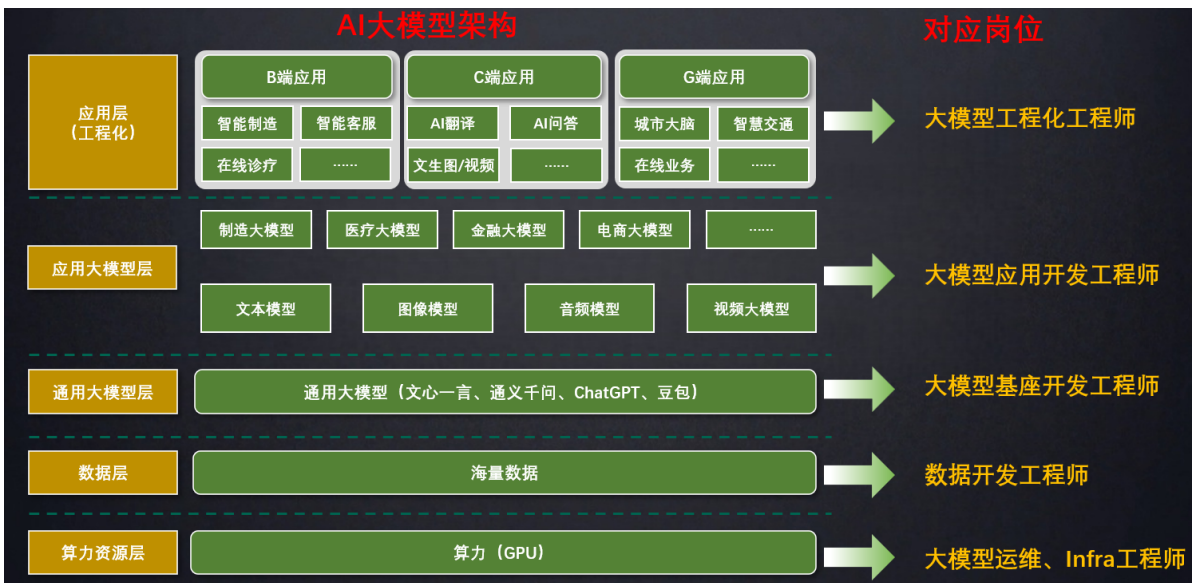
LangChain ≠ LLMs

LangChain 之于 LLMs，类似 Spring 之于 Java

LangChain 之于 LLMs，类似 Django、Flask 之于 Python

顾名思义，LangChain中的“Lang”是指language，即大语言模型，“Chain”即“链”，也就是将大模型与外部数据&各种组件连接成链，以此构建AI应用程序。

大模型相关的岗位



应用开发是大模型最值得关注的方向：应用为王！

学习LangChain框架，高效开发大模型应用

1.2 有哪些大模型应用开发框架呢？

截止到2025年7月26日，GitHub统计数据：

开发语言	开发框架	stars数量	推荐指数
Python	LangChain	112k	★★★★★
Python	LlamaIndex	43.3k	★★★★
Java	LangChain4J	8.5k	★★★
Java	SpringAI	6.2k	★★
Java	SpringAI Alibaba	5.0k	★★
C#	SemanticKernel	25.5k	★★★

- **LangChain**：这些工具里出现最早、最成熟的，适合复杂任务分解和单智能体应用

- **LlamaIndex**：专注于高效的索引和检索，适合 RAG 场景。（注意不是Meta开发的）
- **LangChain4J**：LangChain还出了Java、JavaScript（**LangChain.js**）两个语言的版本，LangChain4j的功能略少于LangChain，但是主要的核心功能都是有的
- **SpringAI/SpringAI Alibaba**：有待进一步成熟，此外只是简单的对于一些接口进行了封装
- **SemanticKernel**：也称为sk，微软推出的，对于C#同学来说，那就是5颗星

1.3 为什么需要LangChain?

问题1：LLMs用的好好的，干嘛还需要LangChain?

在大语言模型（LLM）如 ChatGPT、Claude、DeepSeek 等快速发展的今天，开发者不仅希望能“使用”这些模型，还希望能 **将它们灵活集成到自己的应用中**，实现更强大的对话能力、检索增强生成（RAG）、工具调用（Tool Calling）、多轮推理等功能。



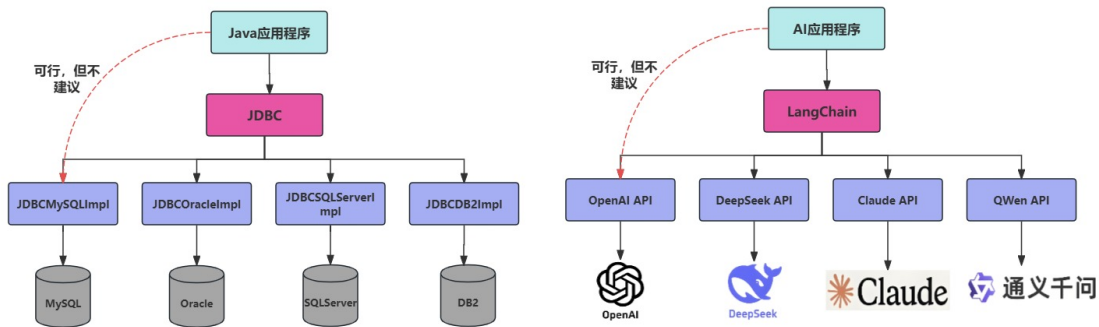
LangChain 为更方便解决这些问题，而生的。比如：大模型默认不能联网，如果需要联网，用 langchain。

问题2：我们可以使用GPT 或GLM4 等模型的API进行开发，为何需要LangChain这样的框架?

不使用LangChain，确实可以使用GPT 或GLM4 等模型的API进行开发。比如，搭建“智能体”（Agent）、问答系统、对话机器人等复杂的 LLM 应用。

但使用LangChain的好处：

- **简化开发难度**：更简单、更高效、效果更好
- **学习成本更低**：不同模型的API不同，调用方式也有区别，切换模型时学习成本高。使用 LangChain，可以以统一、规范的方式进行调用，有更好的移植性。
- **现成的链式组装**：LangChain提供了一些 **现成的链式组装**，用于完成特定的高级任务。让复杂的逻辑变得 **结构化、易组合、易扩展**



问题3: LangChain 提供了哪些功能呢?

LangChain 是一个帮助你构建 LLM 应用的 **全套工具集**。这里涉及到prompt 构建、LLM 接入、记忆管理、工具调用、RAG、智能体开发等模块。

学习 LangChain 最好的方式就是做项目。

1.4 LangChain的使用场景

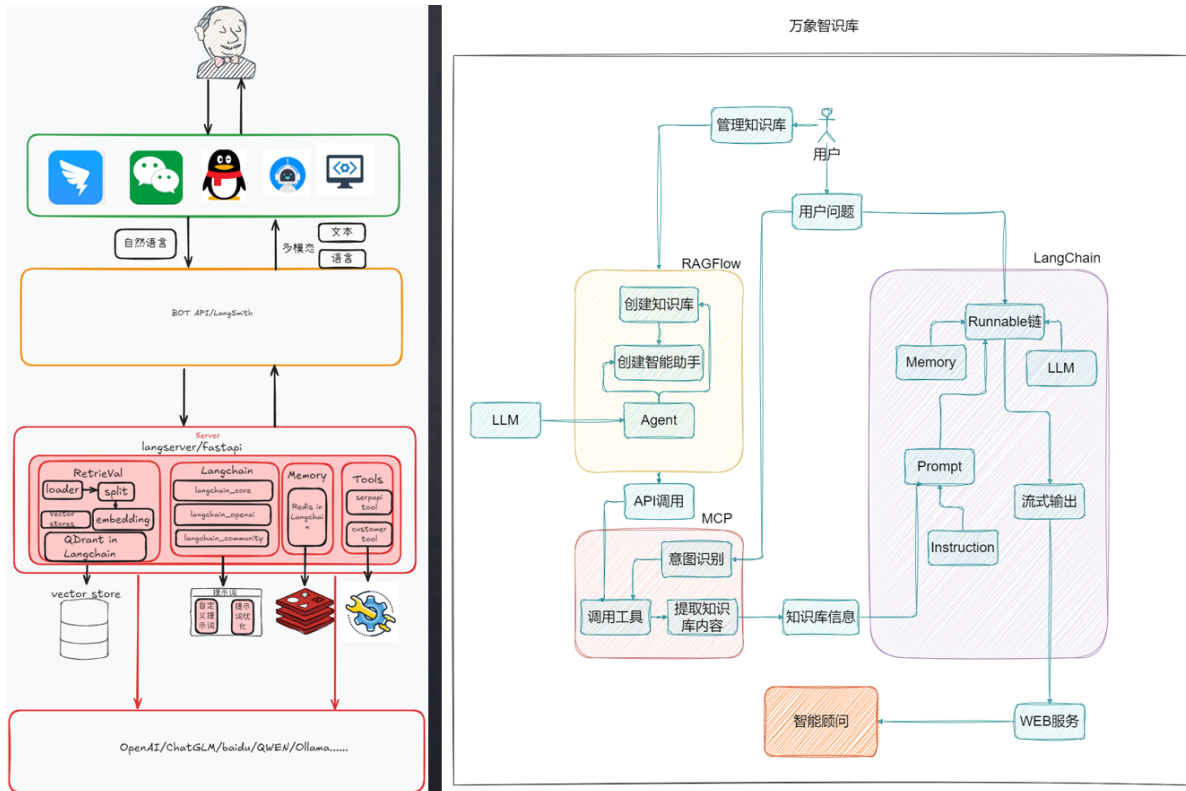
学完LangChain，如下类型的项目，大家都可以实现：

项目名称	技术点	难度
文档问答助手	Prompt + Embedding + RetrievalQA	★★
智能日程规划助手	Agent + Tool + Memory	★★★
LLM+数据库问答	SQLDatabaseToolkit + Agent	★★★★
多模型路由对话系统	RouterChain + 多 LLM	★★★★
互联网智能客服	ConversationChain + RAG +Agent	★★★★★
企业知识库助手（RAG + 本地模型）	VectorDB + LLM + Streamlit	★★★★★

比如：医院智能助手



比如：万象知识库



比如：京东助手



LangChain的位置：



1.5 LangChain资料介绍

官网地址：<https://www.langchain.com/langchain>

官网文档：<https://python.langchain.com/docs/introduction/>

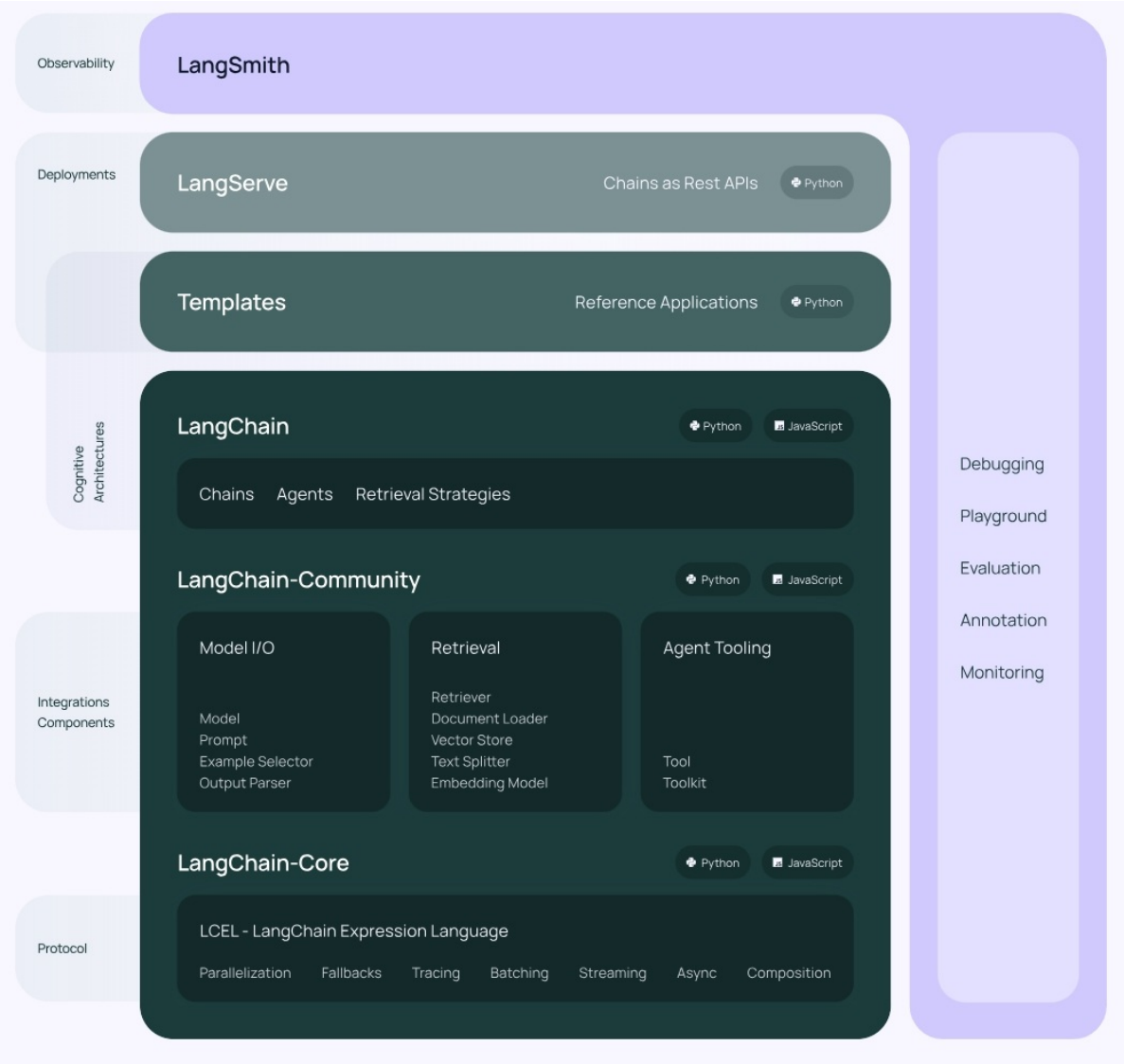
API文档：https://python.langchain.com/api_reference/

github地址：<https://github.com/langchain-ai/langchain>

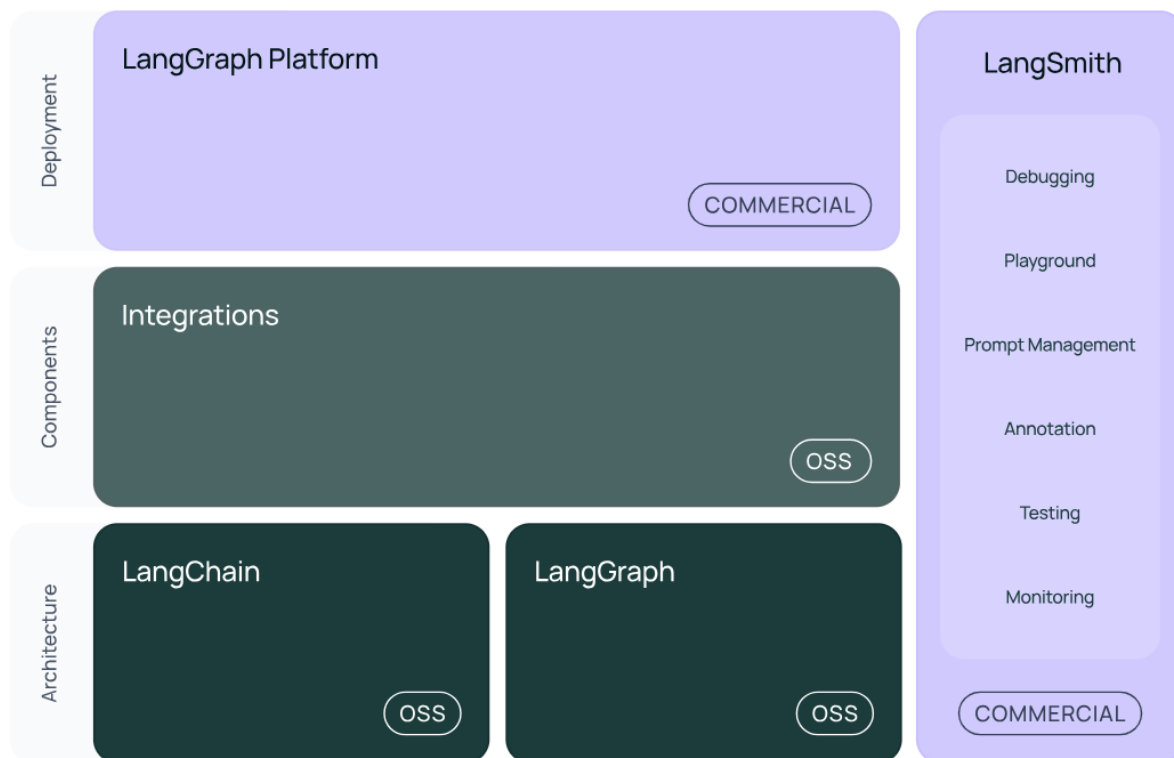
1.6 架构设计

1.6.1 总体架构图

V0.1 版本



V0.2 / V0.3 版本



图中展示了LangChain生态系统的主要组件及其分类，分为三个层次：架构(Architecture)、组件(Components)和部署(Deployment)。

版本的升级，v0.2 相较于v0.1，修改了大概10%-15%。功能性上差不多，主要是往稳定性（或兼容性）、安全性上使劲了，支持更多的大模型，更安全。

1.6.2 内部架构详情

结构1: LangChain

langchain: 构成应用程序认知架构的Chains, Agents, Retrieval strategies等

构成应用程序的链、智能体、RAG。

langchain-community: 第三方集成

比如: Model I/O、Retrieval、Tool & Toolkit; 合作伙伴包 langchain-openai, langchain-anthropic等。

langchain-Core: 基础抽象和LangChain表达式语言 (LCEL)

小结: LangChain, 就是AI应用组装套件, 封装了一堆的API。langchain框架不大, 但是里面琐碎的知识点特别多。就像玩乐高, 提供了很多标准化的乐高零件 (比如, 连接器、轮子等)

结构2: LangGraph

LangGraph可以看做基于LangChain的api的进一步封装, 能够协调多个Chain、Agent、Tools完成更复杂的任务, 实现更高级的功能。

结构3: LangSmith

<https://docs.smith.langchain.com/>

链路追踪。提供了6大功能，涉及Debugging (调试)、Playground (沙盒)、Prompt Management (提示管理)、Annotation (注释)、Testing (测试)、Monitoring (监控)等。与LangChain无缝集成，帮助你从原型阶段过渡到生产阶段。

正是因为LangSmith这样的工具出现，才使得LangChain意义更大，要不仅靠一些API（当然也可以不用，用原生的API），支持不住LangChain的热度。

结构4: LangServe

将LangChain的可运行项和链部署为REST API，使得它们可以通过网络进行调用。

Java怎么调用langchain呢？就通过这个langserve。将langchain应用包装成一个rest api，对外暴露服务。同时，支持更高的并发，稳定性更好。

总结：LangChain当中，最有前途的两个模块就是：LangGraph，LangSmith。

LangChain能做RAG，其它的一些框架也能做，而且做的也不错，比如LlamaIndex。所以这时候LangChain要在Agent这块发力，那就需要LangGraph。而LangSmith，做运维、监控。故，二者是LangChain里最有前途的。

2、开发前的准备工作

2.1 前置知识

1、Python 基础语法

- 变量、函数、类、装饰器、上下文管理器
- 模块导入、包管理（推荐用 `pip` 或 `conda`）

2、大语言模型基础

- 了解什么是 LLM、Token、Prompt、Embedding
- OpenAI API 或其他模型提供商，如 Anthropic、阿里云百炼、DeepSeek等
- 通过浏览器或app使用过大模型（比如：豆包、DeepSeek等）

2.2 相关环境安装

1、安装Python或Anaconda

LangChain基于Python开发，因此需确保系统中安装了Python。

方式1：直接下载Python安装包。推荐版本为Python 3.10及以上

Python官网：<https://www.python.org/>

方式2：使用包管理工具（如Anaconda）进行安装。通过Anaconda可以轻松创建和管理虚拟环境，为项目提供独立的依赖空间，避免不同项目之间的依赖冲突。

具体操作见《[尚硅谷-conda使用指南.md](#)》

2、创建虚拟环境

为了保持项目的独立性与环境的干净，建议使用虚拟环境。可以在Anaconda中创建虚拟环境。

具体操作见《尚硅谷-conda使用指南.md》

验证Python版本：在激活的虚拟环境中，输入以下命令验证Python版本是否正确：

```
1 python --version
```

3、如何下载安装包

比如：安装langchain包

方式1：使用pip指令

基础指令

```
1 # 安装包 (默认最新版)
2 pip install langchain
3
4 # 指定版本
5 pip install langchain==0.3.7
6
7 # 批量安装 (空格分隔)
8 pip install langchain requests numpy
9
10 # 升级包
11 pip install --upgrade langchain
12
13 # 卸载包
14 pip uninstall langchain
15
16 # 查看已安装包
17 pip list
```

高级操作

```
1 # 国内镜像加速 (解决下载慢) -i: 指定镜像源
2 pip install -i https://mirrors.aliyun.com/pypi/simple/ langchain
3
4 # 从本地/URL安装:
5 pip install ./local_package.whl
6 pip install https://github.com/user/repo/archive/main.zip
```

方式2：使用conda指令

```
1 # 安装包 (默认仓库)
2 conda install langchain
3
4 # 指定频道 (如 conda-forge)
```

```

5  conda install -c conda-forge langchain==0.3.7
6
7  # 更新包
8  conda update langchain
9
10 # 卸载包
11 conda uninstall langchain
12
13 # 查看已安装包
14 conda list

```

-c：是 --channel 的缩写，conda用于指定包的安装来源渠道。

conda-forge：该源比官方默认渠道更新更快、包更全

建议：二者最好不好混用，推荐先conda装基础包，后 pip补充的顺序。

- 1 # 检查包来源
- 2 conda list # conda 安装的包显示频道，pip安装的显示 pypi

```

(pyth310) C:\Users\shkst>conda list
# packages in environment at D:\developTools\miniconda3\envs\pyth310:
#
# Name                               Version           Build    Channel
#-----
python_abi3_support                 1.0               hd8ed1ab_2 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aiofiles                            24.1.0            pypi_0   pypi
aiohappyeyeballs                    2.6.1             pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aiohttp                             3.12.2            py310h38315fa_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aiosignal                           1.3.2             pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
altair                              5.5.0             pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
annotated-types                     0.7.0             pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
anthropic                           0.55.0            pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
anyio                                4.9.0             pyh29332c3_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
argon2-cffi                         23.1.0            pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
argon2-cffi-bindings                21.2.0            py310ha8f682b_5 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
arrow                                1.3.0             pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
asttokens                           3.0.0             pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
async-lru                           2.0.5             pyh29332c3_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
async-timeout                       4.0.3             pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
attrs                               25.3.0            pyh71513ae_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-auth                          0.9.0             hd490b63_15 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-cal                           0.9.2             hd8a8e38_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-common                        0.12.3            h2466b09_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-compression                   0.3.1             h5d0e663_5 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-event-stream                  0.5.4             ha416645_12 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-http                          0.10.2            h81282ae_2 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-io                            0.20.1            hddf4d6c_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-mqtt                           0.13.1            h5c1ae27_3 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-s3                            0.8.1             h1e843c7_3 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-c-sdkutils                      0.2.4             h5d0e663_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-checksums                       0.2.7             h5d0e663_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-crt-cpp                         0.32.8            h5d81255_6 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
aws-sdk-cpp                         1.11.510          h7deb975_10 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
babel                               2.17.0            pyhd8ed1ab_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
backoff                             2.2.1            pyhd8ed1ab_1 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
bcrypt                              4.3.0            py310hc226416_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge
beautifulsoup4                      4.13.4            pyha770c72_0 https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge

```

4、PyCharm开发环境

PyCharm作为专业的Python IDE，具有强大的代码编辑、调试和版本控制功能。



创建新的工程，并设置Python解释器（选择Anaconda环境）。

```
1 import langchain
2
3 print(langchain.__version__) # 0.3.25
```

```
1 import openai
2 print(openai.__version__) # 1.81.0
```

```
1 import sys
2
3 #查看python的版本
4 print(sys.version) # 3.10.17 | packaged by conda-forge | (main, Apr 10 2025, 22:06:35) [MSC
v.1943 64 bit (AMD64)]
```

3、大模型应用开发

大模型应用技术特点：门槛低，天花板高。

3.1 基于RAG架构的开发

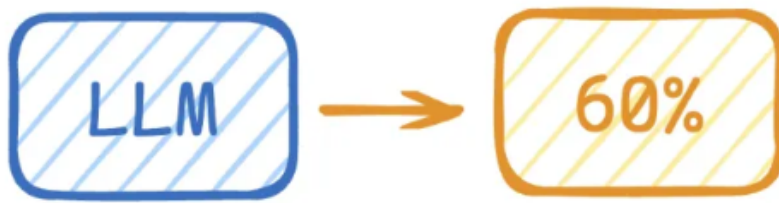
背景：

- 大模型的知识冻结
- 大模型幻觉

而RAG就可以非常精准的解决这两个问题。

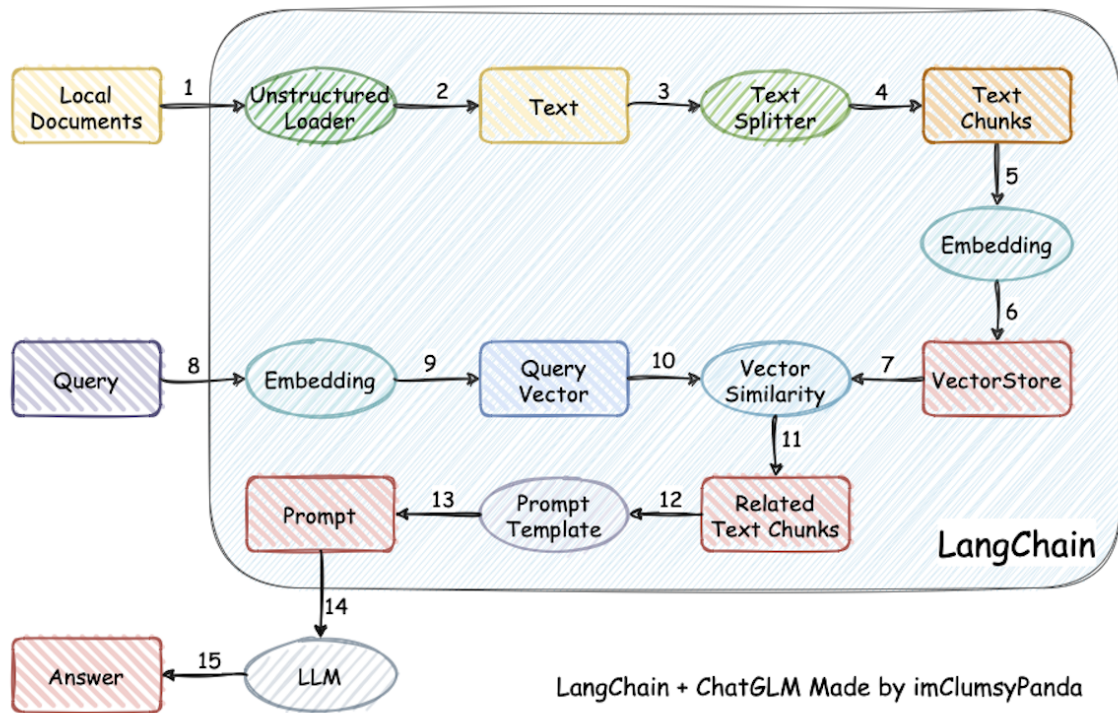
举例：

LLM在考试的时候面对陌生的领域，答复能力有限，然后就准备放飞自我了。而此时RAG给了一些提示和思路，让LLM懂了开始往这个提示的方向做，最终考试的正确率从60%到了90%！



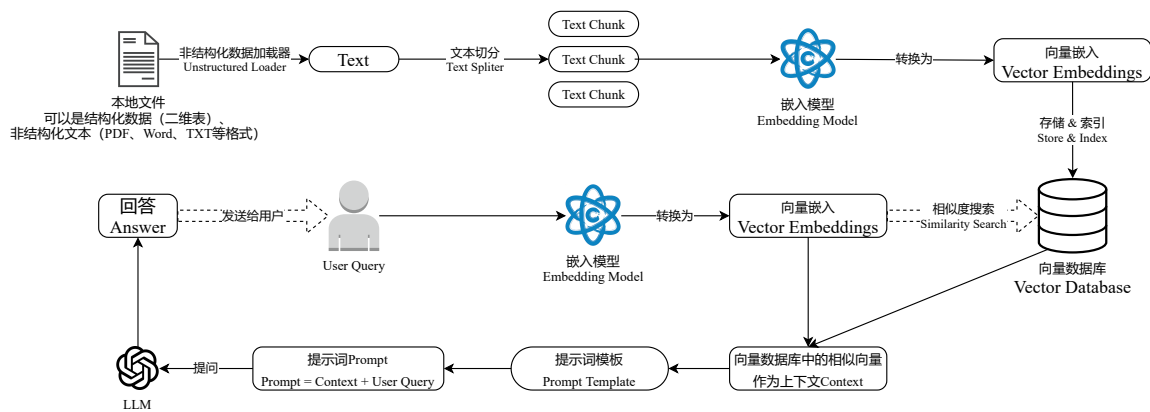
何为RAG?

Retrieval-Augmented Generation (检索增强生成)

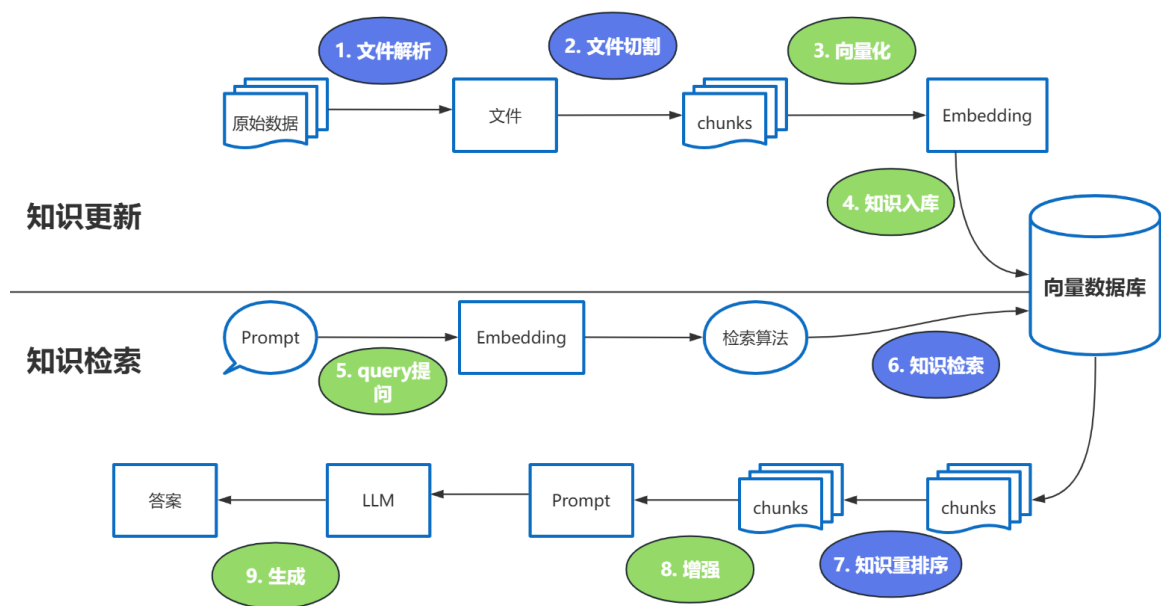


检索-增强-生成过程：检索可以理解为第10步，增强理解为第12步（这里的提示词包含检索到的数据），生成理解为第15步。

类似的细节图：



强调一下难点的步骤：



这些过程中的难点：1、文件解析 2、文件切割 3、知识检索 4、知识重排序

Reranker的使用场景：

- 适合：追求 **回答高精度** 和 **高相关性** 的场景中特别适合使用 Reranker，例如专业知识库或者客服系统等应用。
- 不适合：引入reranker会增加召回时间，增加检索延迟。服务对 **响应时间要求高** 时，使用reranker可能不合适。

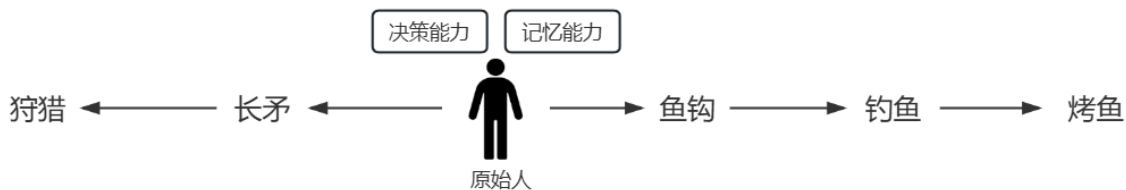
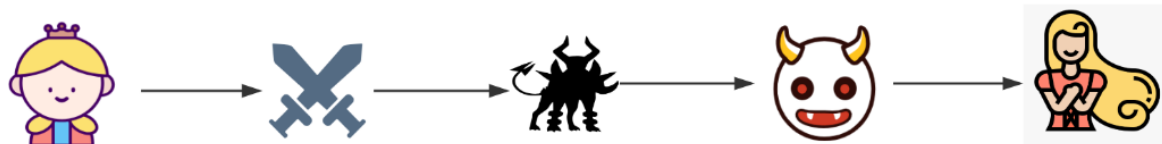
这里有三个位置涉及到大模型的使用：

- 第3步向量化时，需要使用EmbeddingModels。
- 第7步重排序时，需要使用RerankModels。
- 第9步生成答案时，需要使用LLM。

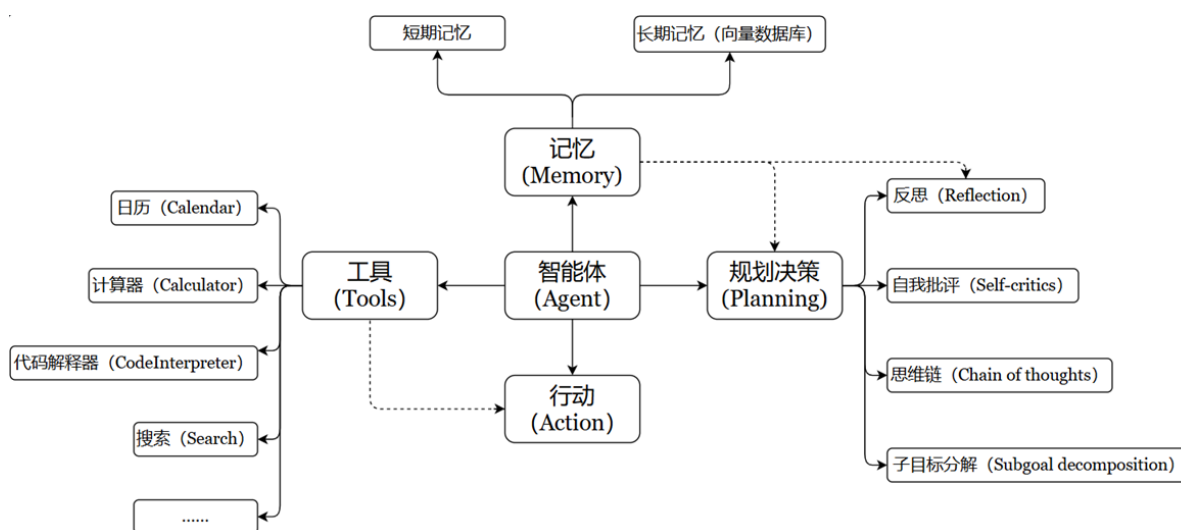
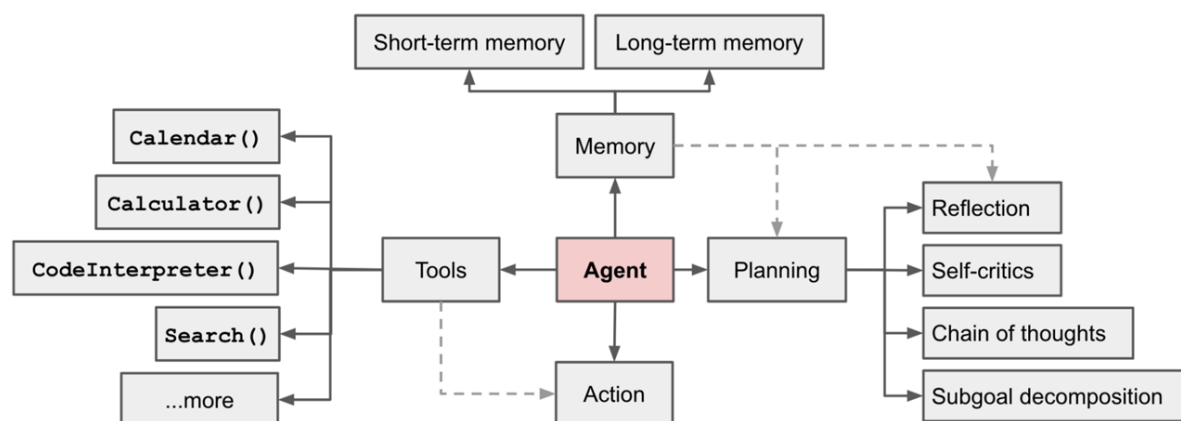
3.2 基于Agent架构的开发

充分利用 LLM 的推理决策能力，通过增加 **规划**、**记忆** 和 **工具** 调用的能力，构造一个能够独立思考、逐步完成给定目标的智能体。

举例：传统的程序 vs Agent（智能体）



OpenAI的元老翁丽莲(Lilian Weng)于2023年6月在个人博客首次提出了现代AI Agent架构。



一个数学公式来表示：

$$\text{Agent} = \text{LLM} + \text{Memory} + \text{Tools} + \text{Planning} + \text{Action}$$

比如，打车到西藏玩。

- 大脑中枢：规划行程的你
- 规划：步骤1：规划打车路线，步骤2：定饭店、酒店，。。。
- 调用工具：调用MCP或FunctionCalling等API，滴滴打车、携程、美团订酒店饭店
- 记忆能力：沟通时，要知道上下文。比如定酒店得知道是西藏路上的酒店，不能聊着聊着忘了最初的目的。
- 能够执行上述操作。说走就走，不能纸上谈兵。

智能体核心要素被细化为以下模块：

1、大模型（LLM）作为“大脑”：提供推理、规划和知识理解能力，是AI Agent的决策中枢。

大脑主要由一个大型语言模型 LLM 组成，承担着信息处理和决策等功能， 并可以呈现推理和规划的过程，能很好地应对未知任务。

2、记忆（Memory）

记忆机制能让智能体在处理重复工作时调用以前的经验，从而避免用户进行大量重复交互。

- 短期记忆：存储单次对话周期的上下文信息，属于临时信息存储机制。受限于模型的上下文窗口长度。



康师傅

02/23 23:12

帮我随机生成40行左右的数据，包括品牌、大区、销售额。其中品牌有安踏、耐克、李宁、阿迪、匹克、鸿星尔克，大区为中国的身份，销售额范围10000 - 100000。每个品牌在多个省份销售。

Tokens: 91



deepseek-ai/DeepSeek-V3 | 硅基流动

02/23 23:12

以下是随机生成的40行数据示例，包括品牌、大区（中国省份）和销售额：

品牌	大区	销售额（元）
安踏	广东	85000
耐克	北京	92000



康师傅

02/23 23:15

上述数据没问题。但是需要按品牌分组，重新排列一下。

Tokens: 25



deepseek-ai/DeepSeek-V3 | 硅基流动

02/23 23:15

好的！以下是按照品牌分组重新排列后的数据示例：

按品牌分组数据

安踏

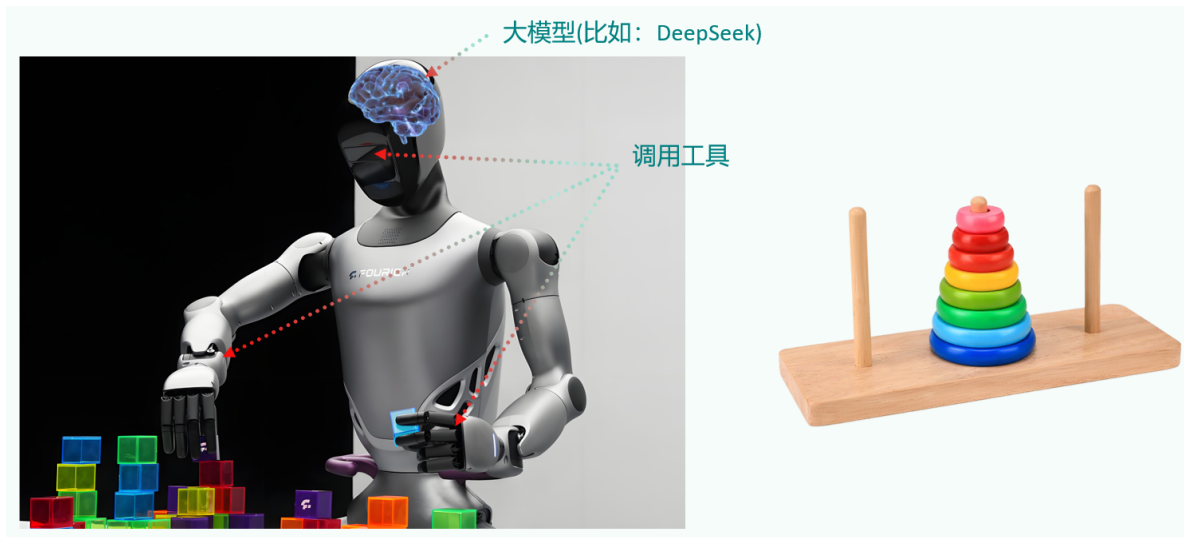
品牌	大区	销售额（元）
安踏	广东	85000

ChatGPT：支持约8k token的上下文
GPT4：支持约32k token的上下文
最新的很多大模型：都支持100万、1000万 token的上下文（相当于2000万字文本或20小时视频）
一般情况下模型中 token 和字数的换算比例大致如下：

- 1 个英文字符 $\approx$ 0.3 个 token。
- 1 个中文字符 $\approx$ 0.6 个 token。

- **长期记忆**：可以横跨多个任务或时间周期，可存储并调用核心知识，非即时任务。
 - 长期记忆，可以通过**模型参数微调（固化知识）**、**知识图谱（结构化语义网络）**或**向量数据库（相似性检索）**方式实现。

3、工具使用 (Tool Use)：调用外部工具（如API、数据库）扩展能力边界。



4、规划决策 (Planning)：通过任务分解、反思与自省框架实现复杂任务处理。例如，利用思维链 (Chain of Thought) 将目标拆解为子任务，并通过反馈优化策略。

秘塔AI搜索

没有广告，直达结果

对比QWQ和DeepSeek-r1|

● 全网 ▾ 长思考·R1 ▾



简洁

深入

研究

对比QWQ和DeepSeek-r1

≡ 研究中...

步骤拆解 ▾

▼ QWQ的定义和背景

QWQ的定义和背景可以从多个角度进行解释：

▼ DeepSeek-r1的定义和背景

▼ ①中的主要功能和技术特点

▼ ②中的主要功能和技术特点

▼ ①中的应用场景和市场定位

▼ ②中的应用场景和市场定位

▼ ①中与②中的性能指标对比

▼ 用户对①中和②中的评价反馈

步骤

QWQ的定义和背景

DeepSeek-r1的定义和背景

①中的主要功能和技术特点

②中的主要功能和技术特点

①中的应用场景和市场定位

②中的应用场景和市场定位

①中与②中的性能指标对比

用户对①中和②中的评价反馈

最终回答

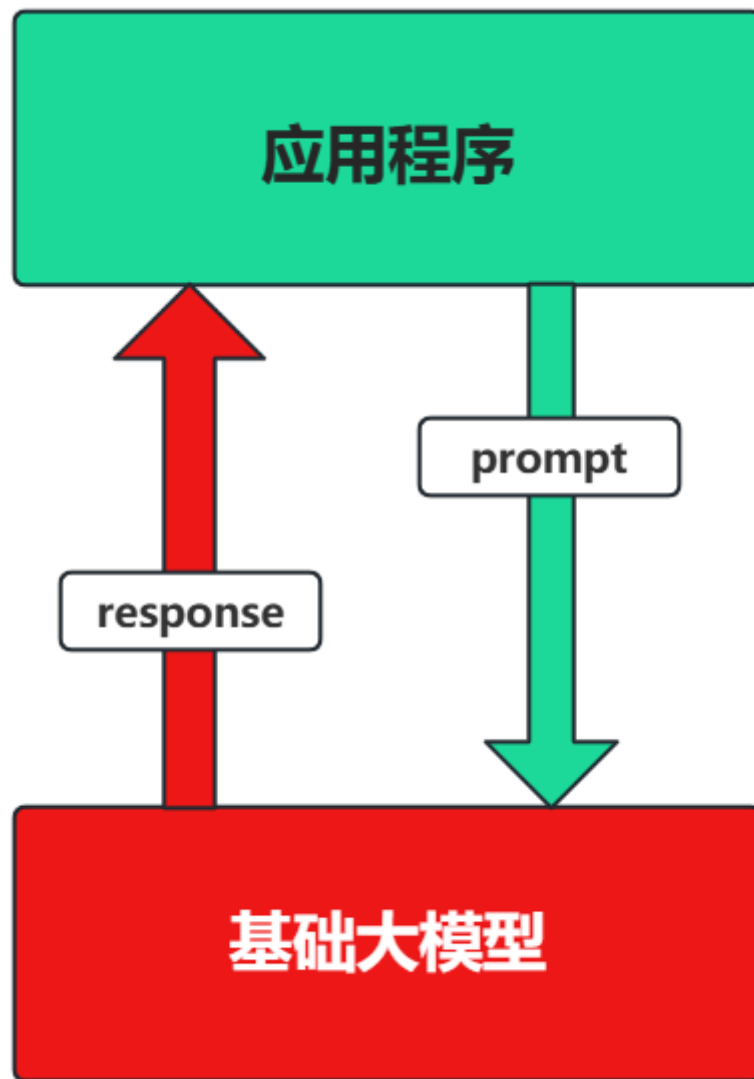
5、行动 (Action)： 实际执行决策的模块，涵盖软件接口操作（如自动订票）和物理交互（如机器人执行搬运）。比如：检索、推理、编程等。

智能体会形成完整的计划流程。例如先读取以前工作的经验和记忆，之后规划子目标并使用相应工具去处理问题，最后输出给用户并完成反思。

3.3 大模型应用开发的4个场景

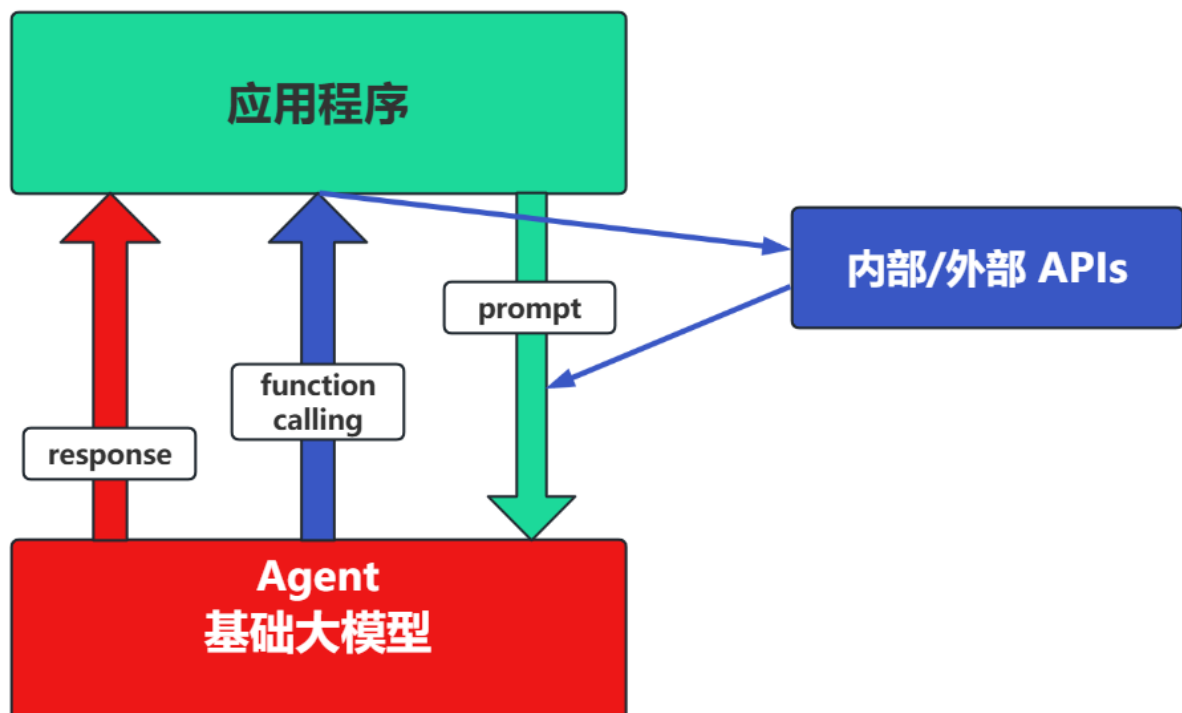
场景1：纯 Prompt

- Prompt是操作大模型的唯一接口
- 当人看：你说一句，ta回一句，你再说一句，ta再回一句...



场景2: Agent + Function Calling

- Agent: AI 主动提要求
- Function Calling: 需要对接外部系统时, AI 要求执行某个函数
- 当人看: 你问 ta「我明天去杭州出差, 要带伞吗? 」, ta 让你先看天气预报, 你看了告诉ta, ta 再告诉你要不要带伞

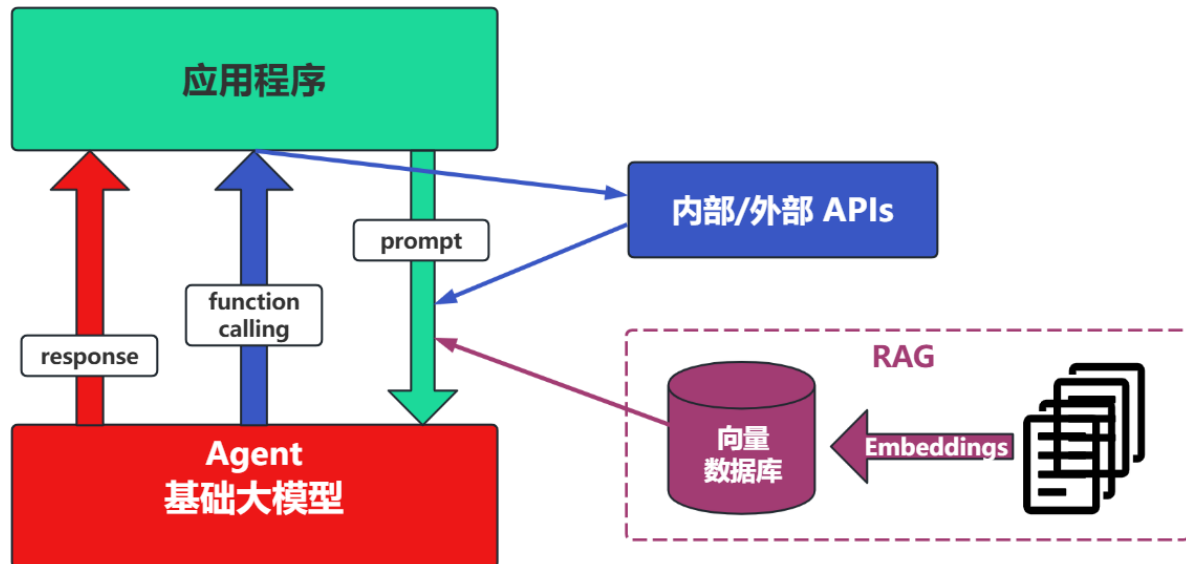


场景3：RAG (Retrieval-Augmented Generation)

RAG：需要补充领域知识时使用

- Embeddings：把文字转换为更易于相似度计算的编码。这种编码叫向量
- 向量数据库：把向量存起来，方便查找
- 向量搜索：根据输入向量，找到最相似的向量

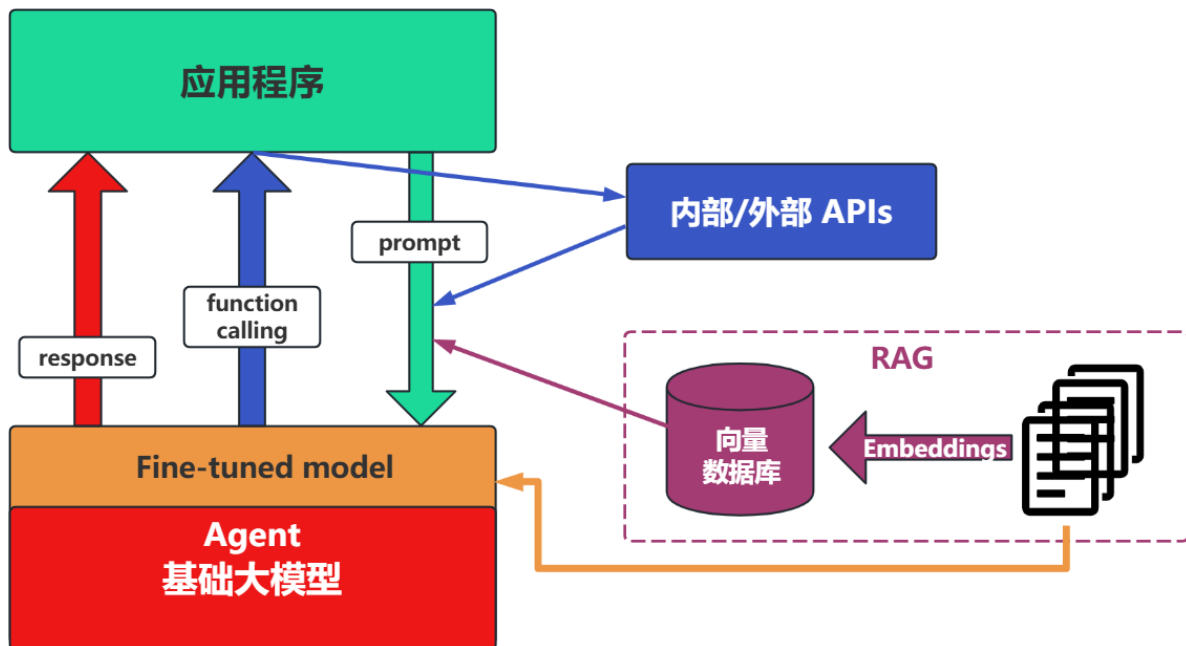
举例：考试答题时，到书上找相关内容，再结合题目组成答案



这个在智能客服上用的最广泛。

场景4：Fine-tuning(精调/微调)

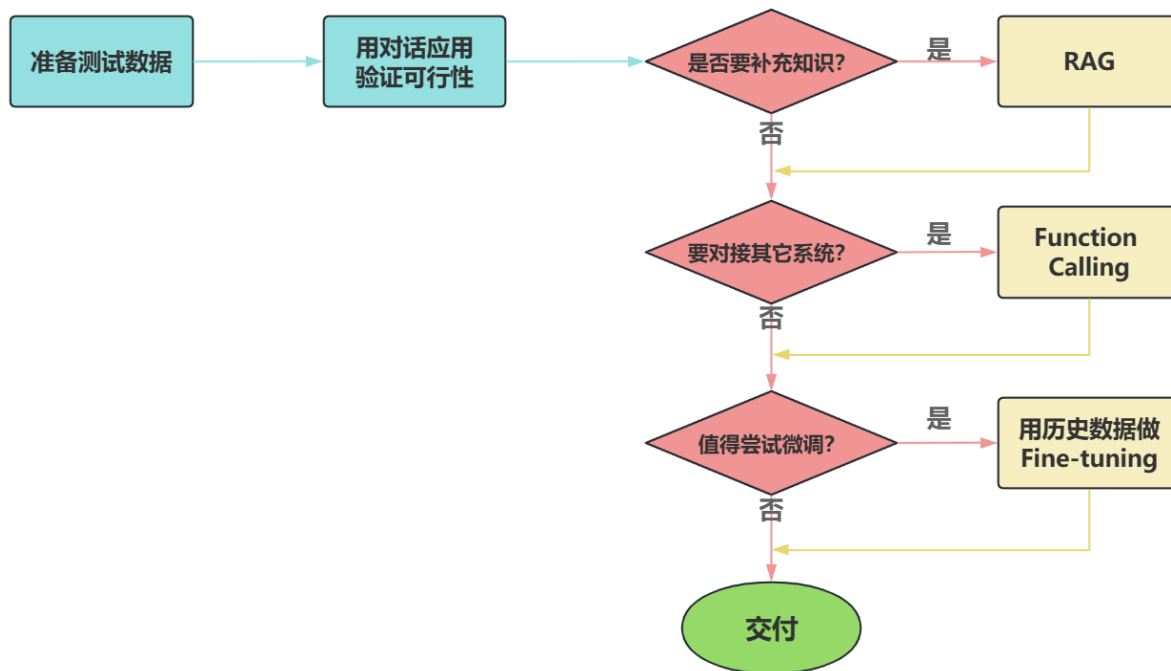
举例：努力学习考试内容，长期记住，活学活用。



特点：成本最高；在前面的方式解决不了问题的情况下，再使用。

如何选择

面对一个需求，如何开始，如何选择技术方案？下面是个常用思路：



注意：其中最容易被忽略的，是准备测试数据

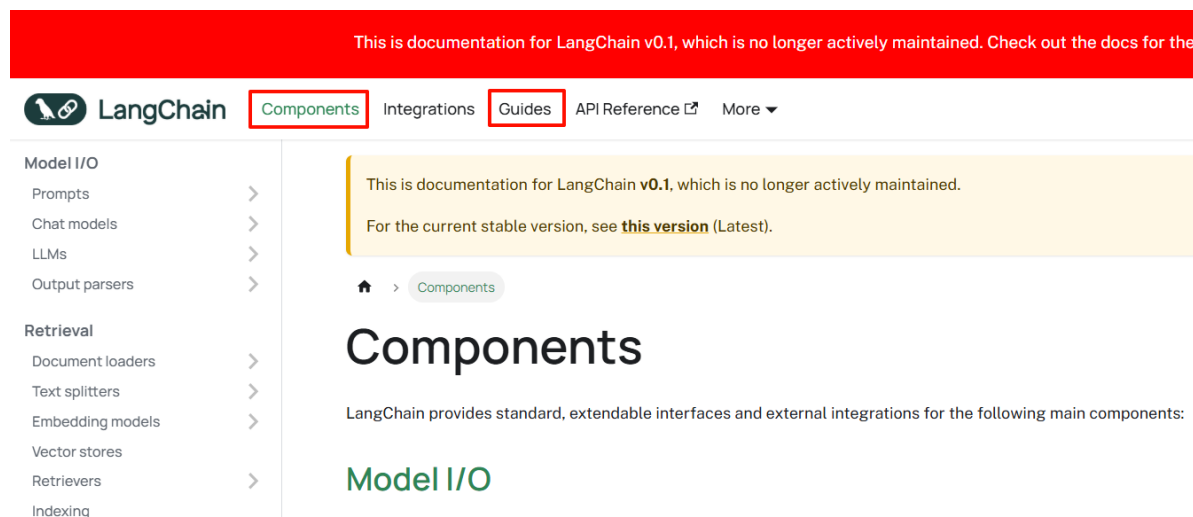
下面，我们重点介绍下大模型应用的开发两类：基于RAG的架构，基于Agent的架构。

4、LangChain的核心组件

学习Langchain最简单直接的方法就是阅读官方文档。

<https://python.langchain.com/v0.1/docs/modules/>

通过文档目录我们可以看到，Langchain构成的核心组件。



两个红框内容是核心。中间的Integrations：集成各种工具或云平台。

4.1 一个问题引发的思考

如果要组织一个AI应用，开发者一般需要什么？

第1，提示词模板的构建，不仅仅只包含用户输入。

第2，模型调用与返回，参数设置，返回内容的格式化输出。

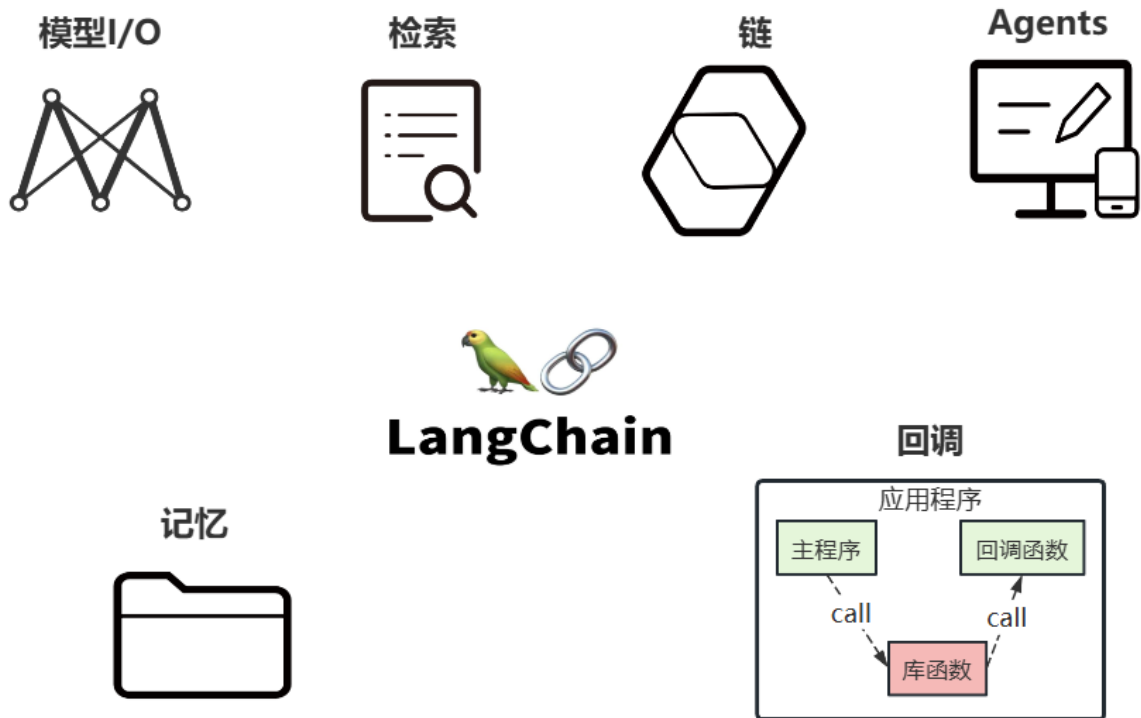
第3，知识库查询，这里会包含文档加载，切割，以及转化为词嵌入（Embedding）向量。

第4，其他第三方工具调用，一般包含天气查询、Google搜索、一些自定义的接口能力调用。

第5，记忆获取，每一个对话都有上下文，在开启对话之前总得获取到之前的上下文吧？

4.2 核心组件的概述

LangChain的核心组件涉及六大模块，这六大模块提供了一个全面且强大的框架，使开发者能够创建复杂、高效且用户友好的基于大模型的应用。



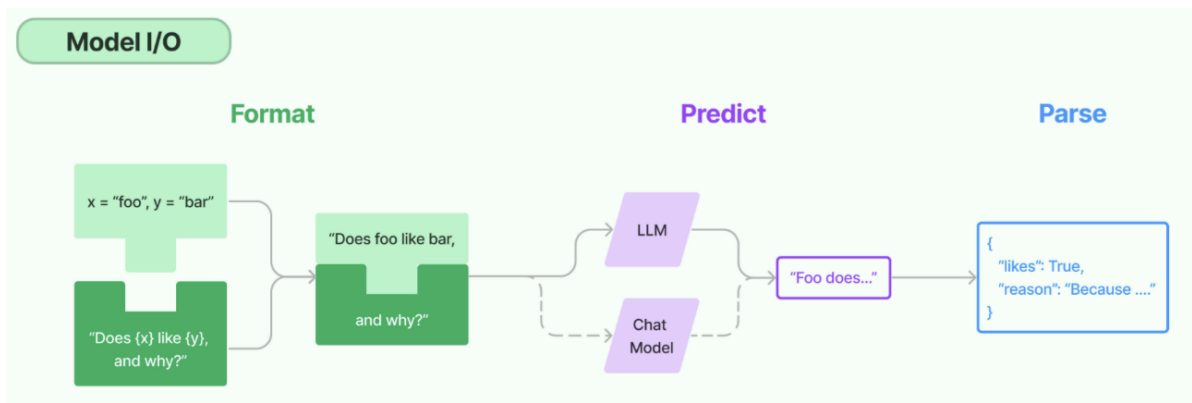
4.3 核心组件的说明

核心组件1：Model I/O

这个模块使用最多，也最简单

Model I/O：标准化各个大模型的输入和输出，包含输入模版，模型本身和格式化输出。

以下是使用语言模型从输入到输出的基本流程。



以下是对每一块的总结：

- **Format(格式化)**：即指代Prompts Template，通过模板管理大模型的输入。将原始数据格式化成模型可以处理的形式，插入到一个模板问题中，然后送入模型进行处理。
- **Predict(预测)**：即指代Models，使用通用接口调用不同的大语言模型。接受被送进来的问题，然后基于这个问题进行预测或生成回答。
- **Parse(生成)**：即指代Output Parser 部分，用来从模型的推理中提取信息，并按照预先设定好的模版来规范化输出。比如，格式化成一个结构化的JSON对象。

核心组件2：Chains

Chain："链条"，用于将多个模块串联起来组成一个完整的流程，是 LangChain 框架中最重要的模块。

例如，一个 Chain 可能包括一个 Prompt 模板、一个语言模型和一个输出解析器，它们一起工作以处理用户输入、生成响应并处理输出。

常见的Chain类型：

- **LLMChain**：最基础的模型调用链
- **SequentialChain**：多个链串联执行
- **RouterChain**：自动分析用户的需求，引导到最适合的链
- **RetrievalQA**：结合向量数据库进行问答的链

核心组件3：Memory

Memory：记忆模块，用于保存对话历史或上下文信息，以便在后续对话中使用。

常见的 Memory 类型：

- **ConversationBufferMemory**：保存完整的对话历史
- **ConversationSummaryMemory**：保存对话内容的精简摘要（适合长对话）
- **ConversationSummaryBufferMemory**：混合型记忆机制，兼具上面两个类型的特点
- **VectorStoreRetrieverMemory**：保存对话历史存储在向量数据库中

核心组件4：Agents

Agents，对应着智能体，是 LangChain 的高阶能力，它可以自主选择工具并规划执行步骤。

Agent 的关键组成：

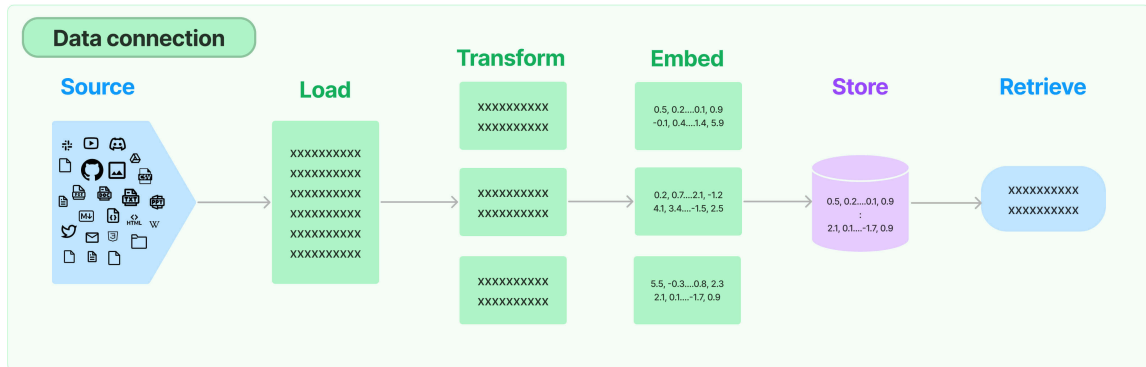
- **AgentType**：定义决策逻辑的工作流模式
- **Tool**：是一些内置的功能模块，如API调用、搜索引擎、文本处理、数据查询等工具。Agents通过这些工具来执行特定的功能。
- **AgentExecutor**：用来运行智能体并执行其决策的工具，负责协调智能体的决策和实际的工具执行。

目前最热门的智能体开发实践，未来能够真正实现通用人工智能的落地方案。

这里的Agent，就会涉及到前面讲的memory，以及tools。

核心组件5: Retrieval

Retrieval：对应着RAG，检索外部数据，然后在执行生成步骤时将其传递到 LLM。步骤包括文档加载、切割、Embedding等



- **Source**：数据源，即大模型可以识别的多种类型的数据：视频、图片、文本、代码、文档等。
- **Load**：负责将来自不同数据源的非结构化数据，加载为文档(Document)对象
- **Transform**：负责对加载的文档进行转换和处理，比如将文本拆分为具有语义意义的小块。
- **Embed**：将文本编码为向量的能力。一种用于嵌入文档，另一种用于嵌入查询
- **Store**：将向量化后的数据进行存储
- **Retrieve**：从大规模文本库中检索和查询相关的文本段落

绿色的是入库存储前的操作。

核心组件6: Callbacks

Callbacks：回调机制，允许连接到 LLM 应用程序的各个阶段，可以监控和分析LangChain的运行情况，比如日志记录、监控、流传输等，以优化性能。

回调函数，对于程序员们应该都不陌生。这个函数允许我们在LLM的各个阶段使用各种各样的“钩子”，从而达实现日志的记录、监控以及流式传输等功能。

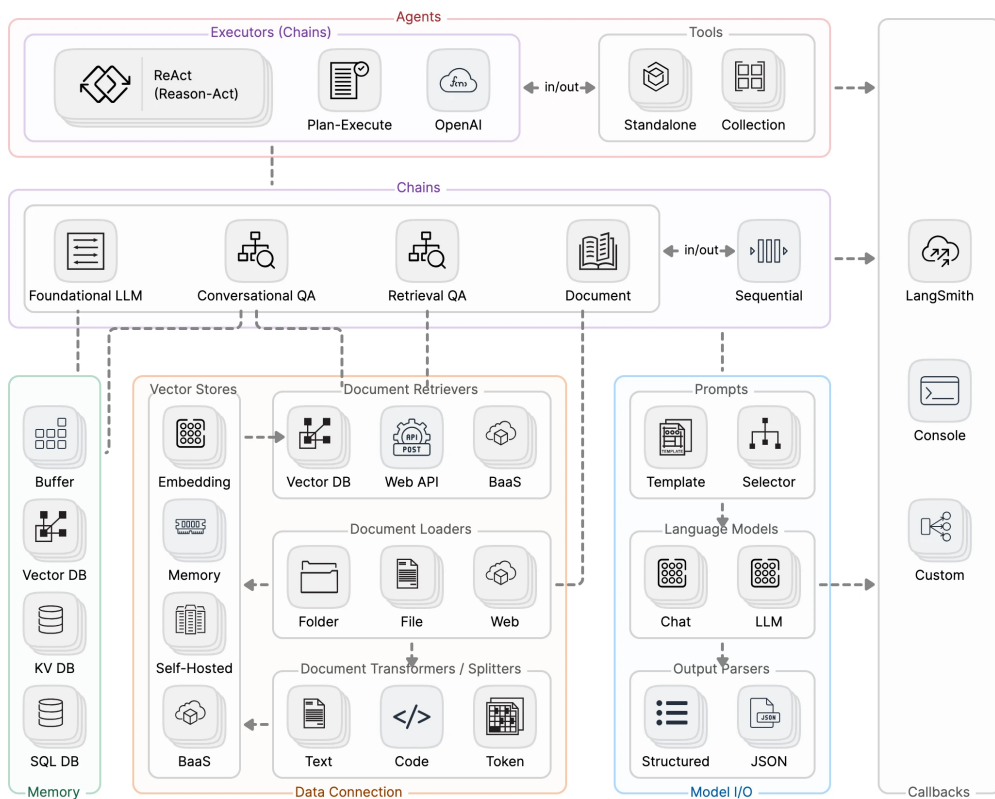
4.4 小结

- Model I/O模块：使用最多，也最简单
- Chains 模块：最重要的模块
- Retrieval模块、Agents模块：大模型的主要落地场景

在这个基础上，其它组件要么是它们的辅助，要么只是完成常规应用程序的任务。

辅助：比如，向量数据库的分块和嵌入，用于追踪、观测的Callbacks

任务：比如，Tools, Memory



LangChain (JS) Modules Overview v0.2.0 for npm:langchain@0.0.114 (Last Updated: July 22, 2023) by @zhanghaoli0610



我们要做的就是一个个module去攻破，最后将他们融会贯通，也就成为一名合格的LangChain学习者了。

5、LangChain的helloworld

注意：大家如果想演示如下代码的话，需要准备两个前提：

- ① 安装必要的库和插件
- ② 参考后续《02-LangChain使用之Model IO》的2.3.2小结，配置好.env配置文件

建议：如下的helloworld大家看看即可，从下一章开始展开讲解

5.1 获取大模型

```

1  #导入 dotenv 库的 load_dotenv 函数，用于加载环境变量文件 (.env) 中的配置
2  import dotenv
3  from langchain_openai import ChatOpenAI
4  import os
5
6  dotenv.load_dotenv() #加载当前目录下的 .env 文件
7
8  os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY1')

```

```

9 os.environ[ 'OPENAI_BASE_URL' ] = os.getenv( "OPENAI_BASE_URL" )
10
11 # 创建大模型实例
12 llm = ChatOpenAI(model= "gpt-4o-mini" ) # 默认使用 gpt-3.5-turbo
13
14 # 直接提供问题，并调用llm
15 response = llm.invoke( "什么是大模型?" )
16 print(response)

```

content='大模型（Large Model）通常是指在深度学习和机器学习领域中，拥有大量参数和复杂结构的模型。这些模型可以处理复杂的数据集，进行高级的任务，如自然语言处理、图像识别、生成对抗网络等。以下是一些关于大模型的关键点：\n\n1. 参数量：大模型通常具备数亿甚至数千亿个参数，这使得它们能够学习到更加丰富的特征和关系。 \n\n2. 计算需求：由于其庞大的规模，大模型通常需要高性能的计算资源，例如GPU或TPU，以及长时间的训练过程。 \n\n3. 数据需求：训练大模型通常需要大量的数据，以确保模型能够在各种情况下进行有效的学习和泛化。 \n\n4. 应用广泛：大模型在许多领域都有应用，包括语言模型（如GPT-3、BERT等）、图像处理（如卷积神经网络）以及生成模型（如GAN）。 \n\n5. 转移学习：许多大模型允许通过转移学习的方式进行微调，使得它们能够在特定任务上表现良好，而无需从头开始训练。 \n\n大模型的出现大大推动了人工智能的发展，使得许多复杂的任务变得可行，并且在多个领域取得了显著的成果。'

```

additional_kwargs={'refusal': None} response_metadata={'token_usage':
{'completion_tokens': 289, 'prompt_tokens': 12, 'total_tokens': 301,
'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0,
'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18',
'system_fingerprint': 'fp_efad92c60b', 'id': 'chatcmpl-BxO81Kq8FpSQgLyTikWfGzOlry2MF',
'service_tier': None, 'finish_reason': 'stop', 'logprobs': None} id='run--134748c7-8c54-4784-
8781-6ea143086285-0' usage_metadata={'input_tokens': 12, 'output_tokens': 289,
'total_tokens': 301, 'input_token_details': {'audio': 0, 'cache_read': 0},
'output_token_details': {'audio': 0, 'reasoning': 0}}

```

其中，需要在当前工程下提供 .env 文件，文件中提供如下信息：

```

1 OPENAI_API_KEY1= "sk-cvUm8OddQbly.....AGgIHtm9kMH7Bf226G2" #你自己的密钥
2 OPENAI_BASE_URL= "https://api.openai-proxy.org/v1" #url是固定值，统一写成这样

```

密钥来自于：<https://www.closeai-asia.com/>


CloseAI

控制台

模型定价

账户管理

开发者模式

秘钥管理

代理商

使用文档

客服工单

个人设置

接口地址

接口地址(备用线路1)

接口地址(备用线路2)

API地址

https://api.openai-proxy.org

状态

正

描述

【主力域名】推荐线上服务使用，全球CDN加速

由于国内各地区宽带运营商不同，部分省市和地区访问海外服务可能存在网络卡顿或者连不上的情况，遇到这类问题可以切换不同线路的域名尝试

秘钥列表

创建

秘钥备注	秘钥	状态
default	sk-7eSHThSRM4EasY4Ks7vyuH8vZjchrM0Q	启用中
langchain_test	sk-cvUm8OddQlJAGgIHtm9kMH7Bf226G2	启用中
test	sk-Lm1sBG6uiHWgdELGh97mGF82HN6	启用中

5.2 使用提示词模板

我们也可以创建prompt template, 并引入一些变量到prompt template中, 这样在应用的时候更加灵活。

```
1 from langchain_core.prompts import ChatPromptTemplate
2
3 # 需要注意的一点是, 这里需要指明具体的role, 在这里是system和用户
4 prompt = ChatPromptTemplate.from_messages([
5     ("system", "你是世界级的技术文档编写者"),
6     ("user", "{input}") # {input}为变量
7 ])
8
9 # 我们可以把prompt和具体llm的调用和在一起。
10 chain = prompt | llm
11 message = chain.invoke({"input": "大模型中的LangChain是什么?"})
12 print(message)
13
14 # print(type(message))
```

- 1 LangChain是一个开源框架, 旨在简化和增强大语言模型 (如GPT-3、GPT-4 等) 的开发与应用。它为开发者提供了一套工具和组件, 使得在不同的用例中利用语言模型变得更加高效和灵活。LangChain 的全名是“Language Chain”, 意在强调它将多个处理步骤或组件串联在一起, 形成一个完整的处理链, 以便更好地利用语言模型的能力。
- 2
- 3 ### LangChain 的主要特性包括:
- 4
- 5 1. **组件化设计**: LangChain 提供了高度模块化的组件, 允许开发者根据需要组合和扩展不同的功能, 比如文本生成、数据提取、工具调用等。
- 6
- 7 2. **链式调用**: 可以将多个操作串联在一起, 例如先生成文本, 然后对生成的内容进行解析或处理, 使得工作流程更加流畅。
- 8
- 9 3. **多种后端支持**: LangChain 支持多种语言模型和生成后端, 方便开发者在不同的平台和环境下进行开发。
- 10
- 11 4. **集成外部工具**: LangChain 允许与外部API、数据库和其他工具集成, 使得模型可以利用外部数据源和功能, 增强其智能性和实用性。
- 12
- 13 5. **可扩展性**: 开发者可以通过自定义组件和方法, 扩展 LangChain 的功能, 以适应特定的业务需求。
- 14
- 15 ### 应用场景:
- 16
- 17 - **对话系统**: 构建更智能的聊天机器人, 能够理解上下文并进行多轮对话。
- 18 - **内容生成**: 生成文章、故事、产品描述等多种内容, 并对其进行编辑和优化。
- 19 - **数据提取和处理**: 从大量文本中提取关键信息, 并将其结构化。
- 20 - **知识库查询**: 结合语言模型与知识库, 提供深度的知识查询和推荐。
- 21
- 22 总之, LangChain 作为大模型应用的一个助手工具, 提升了开发者的生产力, 使得他们可以更专注于构建创新的应用, 而不是陷入底层实现的复杂性中。
- 23

5.3 使用输出解析器

```

1  from langchain_openai import ChatOpenAI
2  from langchain_core.prompts import ChatPromptTemplate
3  from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
4
5  # 初始化模型
6  llm = ChatOpenAI(model="gpt-4o-mini")
7
8  # 创建提示模板
9  prompt = ChatPromptTemplate.from_messages([
10     ("system", "你是世界级的技术文档编写者。"),
11     ("user", "{input}")
12 ])
13
14 # 使用输出解析器
15 # output_parser = StrOutputParser()
16 output_parser = JsonOutputParser()
17
18 # 将其添加到上一个链中
19 # chain = prompt | llm
20 chain = prompt | llm | output_parser
21
22 # 调用它并提出同样的问题。答案是一个字符串，而不是ChatMessage
23 # chain.invoke({"input": "LangChain是什么?"})
24 chain.invoke({"input": "LangChain是什么? 用JSON格式回复, 问题用question, 回答用answer"})

```

```

{'question': 'LangChain是什么?',
 'answer': 'LangChain是一个开源框架，用于构建与大型语言模型（LLMs）交互的应用程序。它提供了一个灵活的工具集，允许开发人员创建具有复杂推理能力、链式任务处理和上下文管理的智能应用。LangChain支持多种数据源和组件，可以帮助开发者在各种环境中更高效地利用语言模型。'}

```

或

```

1  from langchain_openai import ChatOpenAI
2  from langchain_core.prompts import ChatPromptTemplate
3  from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
4
5  # 初始化模型
6  llm = ChatOpenAI(model="gpt-4o-mini")
7
8  # 创建提示模板
9  prompt = ChatPromptTemplate.from_messages([
10     ("system", "你是世界级的技术文档编写者。输出格式要求: {format_instructions}"),
11     ("user", "{input}")
12 ])
13
14 # 使用输出解析器

```

```

15 # output_parser = StrOutputParser()
16 output_parser = JsonOutputParser()
17
18 # 将其添加到上一个链中
19 # chain = prompt | llm
20 chain = prompt | llm | output_parser
21
22 # 调用它并提出同样的问题。答案是一个字符串，而不是ChatMessage
23 # chain.invoke({"input": "LangChain是什么?"})
24 chain.invoke({"input": "LangChain是什么?", "format_instructions": output_parser.get_format_instructions()})

```

```

{'LangChain': {'description': 'LangChain是一个开源的框架，用于构建基于语言模型的应用程序。它旨在将不同的语言模型、数据源和工具结合起来，以便于开发者创建复杂的应用。',
'key_features': ['模块化设计：支持将不同的组件组合在一起，如语言模型、数据处理和外部API。',
'支持多种语言模型：兼容不同类型的语言模型，如OpenAI的GPT系列、BERT等。',
'内置数据连接：可以与多种数据源进行集成，例如数据库、文件和外部API。',
'灵活的工作流管理：提供了多种方法来定义和管理任务的执行流程。',
'社区支持：拥有活跃的社区，提供文档、示例和支持。',
'可扩展性：允许开发者根据需求定制和扩展功能。'],
'use_cases': ['聊天机器人开发', '文本生成和摘要', '内容推荐系统', '自然语言理解任务', '自动化工作流程处理'],
'getting_started': {'installation': '使用pip安装： pip install langchain',
'documentation_link': 'https://langchain.readthedocs.io/en/latest/'}}

```

5.4 使用向量存储

使用一个简单的本地向量存储 FAISS，首先需要安装它

```

1 pip install faiss-cpu
2 #或者
3 conda install faiss-cpu

```

```

1 pip install langchain_community==0.3.7
2 #或者
3 conda install langchain_community==0.3.7

```

```

1 # 导入和使用 WebBaseLoader
2 from langchain_community.document_loaders import WebBaseLoader
3 import bs4
4
5 loader = WebBaseLoader(
6     web_path="https://www.gov.cn/xinwen/2020-06/01/content_5516649.htm",
7     bs_kwargs=dict(parse_only=bs4.SoupStrainer(id="UCAP-CONTENT"))
8 )
9 docs = loader.load()
10 # print(docs)
11
12 # 对于嵌入模型，这里通过 API调用

```

```

13 from langchain_openai import OpenAIEmbeddings
14
15 embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
16
17
18 from langchain_community.vectorstores import FAISS
19 from langchain_text_splitters import RecursiveCharacterTextSplitter
20
21 # 使用分割器分割文档
22 text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
23 documents = text_splitter.split_documents(docs)
24 print(len(documents))
25 # 向量存储 embeddings 会将 documents 中的每个文本片段转换为向量，并将这些向量存储在 FAISS
    向量数据库中
26 vector = FAISS.from_documents(documents, embeddings)

```

248

5.5 RAG(检索增强生成)

基于外部知识，增强大模型回复

```

1 from langchain_core.prompts import PromptTemplate
2
3 retriever = vector.as_retriever()
4 retriever.search_kwargs = {"k": 3}
5 docs = retriever.invoke("建设用地使用权是什么? ")
6
7 # for i,doc in enumerate(docs):
8 #     print(f"★第{i+1}条规定: ")
9 #     print(doc)
10
11 # 6.定义提示词模版
12 prompt_template = """
13 你是一个问答机器人。
14 你的任务是根据下述给定的已知信息回答用户问题。
15 确保你的回复完全依据下述已知信息。不要编造答案。
16 如果下述已知信息不足以回答用户的问题，请直接回复"我无法回答您的问题"。
17
18 已知信息:
19 {info}
20
21 用户问:
22 {question}
23
24 请用中文回答用户问题。
25 """
26 # 7.得到提示词模版对象
27 template = PromptTemplate.from_template(prompt_template)
28
29 # 8.得到提示词对象
30 prompt = template.format(info=docs, question="建设用地使用权是什么? ")
31

```

```
32 ## 9. 调用LLM
33 response = llm.invoke(prompt)
34 print(response.content)
```

建设用地使用权是指在法律规定的范围内，建设用地使用权人对特定土地的使用、建造建筑物及其他设施的权利。根据相关法律规定，建设用地使用权可以通过出让或划拨等方式设立，并应当符合资源节约、生态环境保护及土地用途的规定。建设用地使用权人有权转让、出资、赠与或抵押其使用权，但需遵循法律规定和合同约定。

5.6 使用Agent

```
1  from langchain.tools.retriever import create_retriever_tool
2
3  # 检索器工具
4  retriever_tool = create_retriever_tool(
5      retriever,
6      "CivilCodeRetriever",
7      "搜索有关中华人民共和国民法典的信息。关于中华人民共和国民法典的任何问题，您必须使用此工具!",
8  )
9
10 tools = [retriever_tool]
11
12 from langchain import hub
13 from langchain.agents import create_openai_functions_agent
14 from langchain.agents import AgentExecutor
15
16 # https://smith.langchain.com/hub
17 prompt = hub.pull("hwchase17/openai-functions-agent")
18
19 agent = create_openai_functions_agent(llm, tools, prompt)
20 agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
21
22 # 运行代理
23 agent_executor.invoke({"input": "建设用地使用权是什么"})
```

```
1  > Entering new AgentExecutor chain...
2
3  Invoking: `CivilCodeRetriever` with `{'query': '建设用地使用权'}`
4
5
6  > Entering new AgentExecutor chain...
7
8  Invoking: `CivilCodeRetriever` with `{'query': '建设用地使用权'}`
9
10
11  第三百四十五条 建设用地使用权可以在土地的地表、地上或者地下分别设立。
12  第三百四十六条 设立建设用地使用权，应当符合节约资源、保护生态环境的要求，遵守法律、
    行政法规关于土地用途的规定，不得损害已经设立的用益物权。
13  第三百四十七条 设立建设用地使用权，可以采取出让或者划拨等方式。
```

14 工业、商业、旅游、娱乐和商品住宅等经营性用地以及同一土地有两个以上意向用地者的，应当采取招标、拍卖等公开竞价的方式出让。

15 严格限制以划拨方式设立建设用地使用权。

16 第三百四十八条 通过招标、拍卖、协议等出让方式设立建设用地使用权的，当事人应当采用书面形式订立建设用地使用权出让合同。

17 建设用地使用权出让合同一般包括下列条款：

18 （一）当事人的名称和住所；

19 （二）土地界址、面积等；

20 （三）建筑物、构筑物及其附属设施占用的空间；

21 （四）土地用途、规划条件；

22 （五）建设用地使用权期限；

23 （六）出让金等费用及其支付方式；

24 （七）解决争议的方法。

25 第三百四十九条 设立建设用地使用权的，应当向登记机构申请建设用地使用权登记。建设用地使用权自登记时设立。登记机构应当向建设用地使用权人发放权属证书。

26

27 第三百五十条 建设用地使用权人应当合理利用土地，不得改变土地用途；需要改变土地用途的，应当依法经有关行政主管部门批准。

28 第三百五十一条 建设用地使用权人应当依照法律规定以及合同约定支付出让金等费用。

29 第三百五十二条 建设用地使用权人建造的建筑物、构筑物及其附属设施的所有权属于建设用地使用权人，但是有相反证据证明的除外。

30 第三百五十三条 建设用地使用权人有权将建设用地使用权转让、互换、出资、赠与或者抵押，但是法律另有规定的除外。

31 第三百五十四条 建设用地使用权转让、互换、出资、赠与或者抵押的，当事人应当采用书面形式订立相应的合同。使用期限由当事人约定，但是不得超过建设用地使用权的剩余期限。

32 第三百五十五条 建设用地使用权转让、互换、出资或者赠与的，应当向登记机构申请变更登记。

33 第三百五十六条 建设用地使用权转让、互换、出资或者赠与的，附着于该土地上的建筑物、构筑物及其附属设施一并处分。

34 第三百五十七条 建筑物、构筑物及其附属设施转让、互换、出资或者赠与的，该建筑物、构筑物及其附属设施占用范围内的建设用地使用权一并处分。

35

36 第三百五十八条 建设用地使用权期限届满前，因公共利益需要提前收回该土地的，应当依据本法第二百四十三条的规定对该土地上的房屋以及其他不动产给予补偿，并退还相应的出让金。

37 第三百五十九条 住宅建设用地使用权期限届满的，自动续期。续期费用的缴纳或者减免，依照法律、行政法规的规定办理。

38 非住宅建设用地使用权期限届满后的续期，依照法律规定办理。该土地上的房屋以及其他不动产的归属，有约定的，按照约定；没有约定或者约定不明确的，依照法律、行政法规的规定办理。

39 第三百六十条 建设用地使用权消灭的，出让人应当及时办理注销登记。登记机构应当收回权属证书。

40 第三百六十一条 集体所有的土地作为建设用地的，应当依照土地管理的法律规定办理。

41 第十三章 宅基地使用权

42 第三百六十二条 宅基地使用权人依法对集体所有的土地享有占有和使用的权利，有权依法利用该土地建造住宅及其附属设施。

43 第三百六十三条 宅基地使用权的取得、行使和转让，适用土地管理的法律和国家有关规定。

44 > Entering new AgentExecutor chain...

45

46 Invoking: `CivilCodeRetriever` with `{ 'query': '建设用地使用权' }

47

48

49 第三百四十五条 建设用地使用权可以在土地的地表、地上或者地下分别设立。

50 第三百四十六条 设立建设用地使用权，应当符合节约资源、保护生态环境的要求，遵守法律、行政法规关于土地用途的规定，不得损害已经设立的用益物权。

51 第三百四十七条 设立建设用地使用权，可以采取出让或者划拨等方式。

52 工业、商业、旅游、娱乐和商品住宅等经营性用地以及同一土地有两个以上意向用地者的，应当采取招标、拍卖等公开竞价的方式出让。

53 严格限制以划拨方式设立建设用地使用权。

54 第三百四十八条 通过招标、拍卖、协议等出让方式设立建设用地使用权的，当事人应当采用书面形式订立建设用地使用权出让合同。

55 建设用地使用权出让合同一般包括下列条款：

56 （一）当事人的名称和住所；

57 （二）土地界址、面积等；

58 （三）建筑物、构筑物及其附属设施占用的空间；

59 （四）土地用途、规划条件；

60 （五）建设用地使用权期限；

61 （六）出让金等费用及其支付方式；

62 （七）解决争议的方法。

63 第三百四十九条 设立建设用地使用权的，应当向登记机构申请建设用地使用权登记。建设用地使用权自登记时设立。登记机构应当向建设用地使用权人发放权属证书。

64

65 第三百五十条 建设用地使用权人应当合理利用土地，不得改变土地用途；需要改变土地用途的，应当依法经有关行政主管部门批准。

66 第三百五十一条 建设用地使用权人应当依照法律规定以及合同约定支付出让金等费用。

67 第三百五十二条 建设用地使用权人建造的建筑物、构筑物及其附属设施的所有权属于建设用地使用权人，但是有相反证据证明的除外。

68 第三百五十三条 建设用地使用权人有权将建设用地使用权转让、互换、出资、赠与或者抵押，但是法律另有规定的除外。

69 第三百五十四条 建设用地使用权转让、互换、出资、赠与或者抵押的，当事人应当采用书面形式订立相应的合同。使用期限由当事人约定，但是不得超过建设用地使用权的剩余期限。

70 第三百五十五条 建设用地使用权转让、互换、出资或者赠与的，应当向登记机构申请变更登记。

71 第三百五十六条 建设用地使用权转让、互换、出资或者赠与的，附着于该土地上的建筑物、构筑物及其附属设施一并处分。

72 第三百五十七条 建筑物、构筑物及其附属设施转让、互换、出资或者赠与的，该建筑物、构筑物及其附属设施占用范围内的建设用地使用权一并处分。

73

74 第三百五十八条 建设用地使用权期限届满前，因公共利益需要提前收回该土地的，应当依照本法第二百四十三条的规定对该土地上的房屋以及其他不动产给予补偿，并退还相应的出让金。

75 第三百五十九条 住宅建设用地使用权期限届满的，自动续期。续期费用的缴纳或者减免，依照法律、行政法规的规定办理。

76 非住宅建设用地使用权期限届满后的续期，依照法律规定办理。该土地上的房屋以及其他不动产的归属，有约定的，按照约定；没有约定或者约定不明确的，依照法律、行政法规的规定办理。

77 第三百六十条 建设用地使用权消灭的，出让人应当及时办理注销登记。登记机构应当收回权属证书。

78 第三百六十一条 集体所有的土地作为建设用地的，应当依照土地管理的法律规定办理。

79 第十三章 宅基地使用权

80 第三百六十二条 宅基地使用权人依法对集体所有的土地享有占有和使用的权利，有权依法利用该土地建造住宅及其附属设施。

81 第三百六十三条 宅基地使用权的取得、行使和转让，适用土地管理的法律和国家有关规定。

82 第三百六十四条 宅基地因自然灾害等原因灭失的，宅基地使用权消灭。对失去宅基地的村民，应当依法重新分配宅基地。建设用地使用权是指在中华人民共和国法律框架下，个人或单位依法享有的对特定建设用地进行开发、建设和利用的权利。

83

84 根据《中华人民共和国民法典》，建设用地使用权的相关规定主要包括以下几个方面：

85

86 1. **设立与形式**：建设用地使用权可以通过出让或划拨等方式设立，通常需要通过招标、拍卖等公开方式进行。

87

88 2. ****合同与登记****：设立建设用地使用权时，需要书面订立相关合同，并向登记机构申请登记。只有登记后，建设用地使用权才正式成立，并会获得权属证书。

89

90 3. ****权利与义务****：建设用地使用权人有权合理利用土地，但不得随意改变土地用途，任何用途的改变都需获得相关主管部门的批准。此外，建设用地使用权人应按照合同约定支付出让金等费用。

91

92 4. ****转让与变更****：建设用地使用权可以进行转让、互换、出资、赠与或抵押，但必须遵循法律规定，并进行相应的登记。

93

94 5. ****期满续期****：住宅建设用地使用权在期满后会自动续期，而非住宅建设用地的续期需依照法律规定办理。

95

96 这些规定旨在确保土地资源的合理利用与保护，同时维护用地权利人的合法权益。

97

98 > Finished chain.

```
1 {'input': '建设用地使用权是什么',
2  'output': '建设用地使用权是指在中华人民共和国法律框架下，个人或单位依法享有的对特定建设
  用地进行开发、建设和利用的权利。\\n\\n根据《中华人民共和国民法典》，建设用地使用权的相关规定
  主要包括以下几个方面：\\n\\n1. **设立与形式**：建设用地使用权可以通过出让或划拨等方式
  设立，通常需要通过招标、拍卖等公开方式进行。\\n\\n2. **合同与登记**：设立建设用地使用权
  时，需要书面订立相关合同，并向登记机构申请登记。只有登记后，建设用地使用权才正式成立，
  并会获得权属证书。\\n\\n3. **权利与义务**：建设用地使用权人有权合理利用土地，但不得随意改
  变土地用途，任何用途的改变都需获得相关主管部门的批准。此外，建设用地使用权人应按照合同
  约定支付出让金等费用。\\n\\n4. **转让与变更**：建设用地使用权可以进行转让、互换、出资、赠
  与或抵押，但必须遵循法律规定，并进行相应的登记。\\n\\n5. **期满续期**：住宅建设用地使用权
  在期满后会自动续期，而非住宅建设用地的续期需依照法律规定办理。\\n\\n这些规定旨在确保土地
  资源的合理利用与保护，同时维护用地权利人的合法权益。'}
}
```